

容器云PaaS层设计&实践

营销事业部

王宇

CONTENTS

目录

01

容器云

02

PaaS层设计实践

03

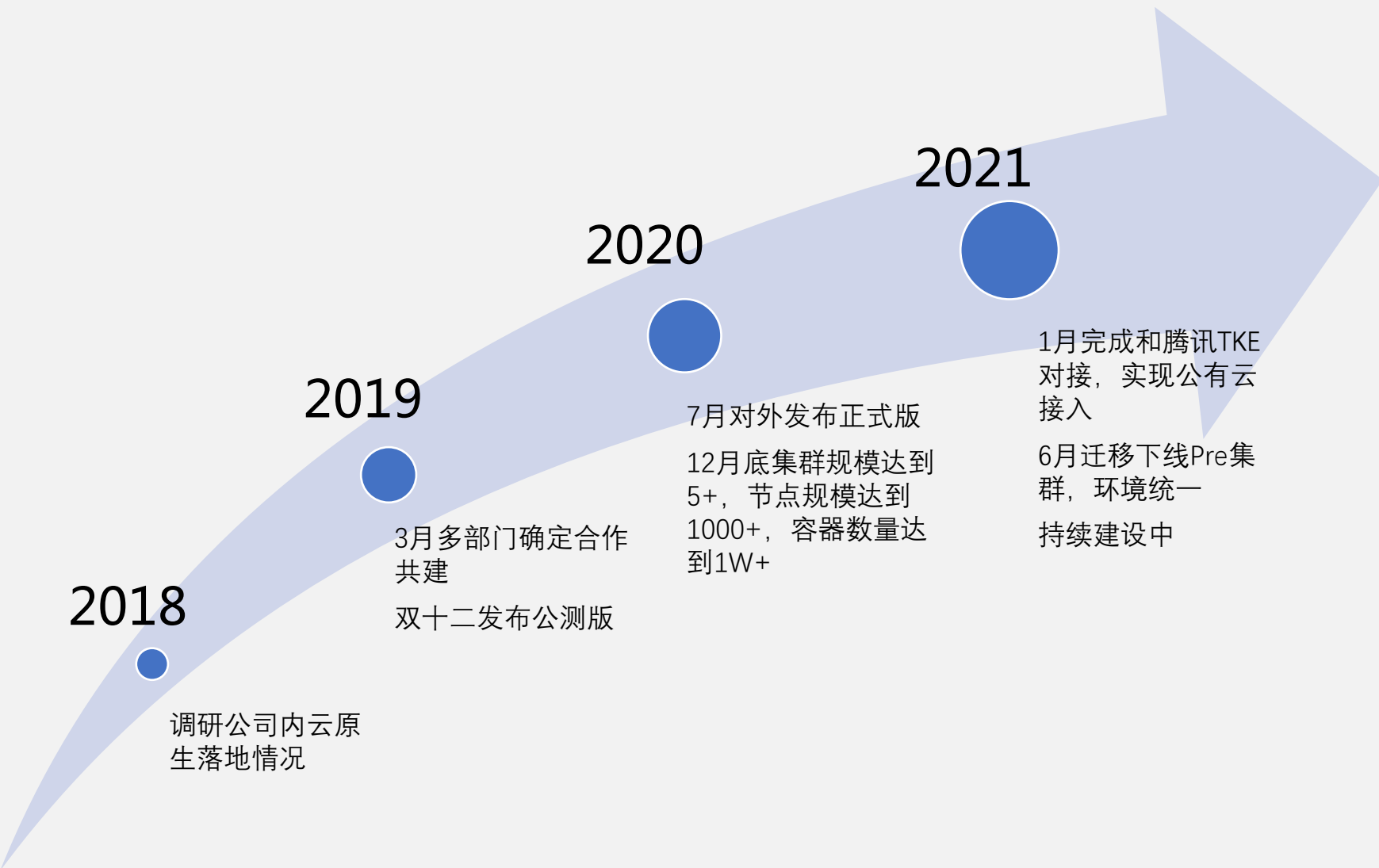
总结

01

容器云

容器云

发展历程



容器云是搜狗内部多部门协作自研的私有云平台。提供贯穿开发、测试、运维等应用生命周期的平台一体化服务；提供微服务框架，兼容主流云生态，实现快速构建基于微服务架构的分布式应用

容器云

面临的挑战



环境复杂性



业务低峰期资源利用率低



弹性伸缩能力不足



容器云

如何构建

- 云原生技术：k8s , docker
- 云平台=K8S+Docker ?
 - 网络、存储如何选型？
 - 持续集成、服务发现、服务授权？
 - 已有开发、测试、运维流程如何对接？
 - 自动化平台如何实现？
 - 如何业务适配？
- 私有云建设



容器云

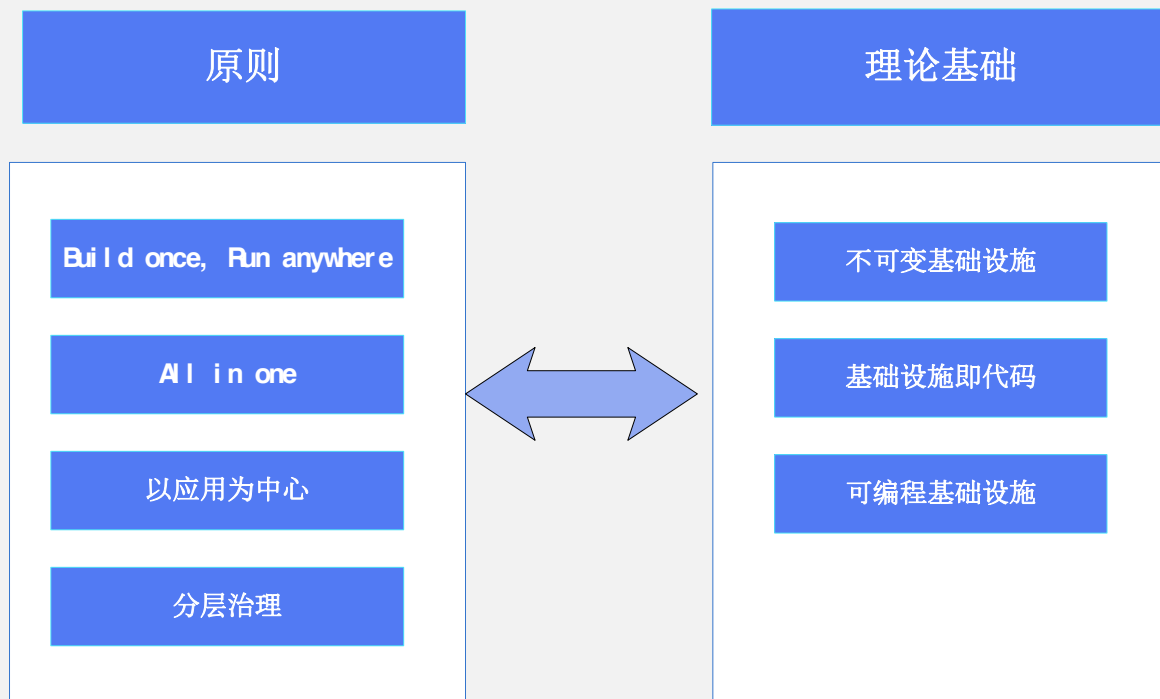
构建的理论基础和原则

● 理论基础

- ✓ 不可变基础设施，是利用Docker镜像的不可变性更加便捷的方式维护基础设施，应用的基础设施应该是不可变的，是一个自包含、自描述可以完全在不同环境中迁移的东西
- ✓ 可编排的资源，主要包含应用、节点、服务等资源的定义、调度、分配、管理

● 原则

- ✓ Build Once, Run anywhere，一个Docker镜像贯穿DEV、QA到生产的各个环节，保持镜像的一致
- ✓ 要求每个Docker镜像中只运行一个Web App，包含所有依赖
- ✓ 以应用为中心，确保关注重点是应用，优雅地管理应用的生命周期
- ✓ 分层治理，通过SaaS、PaaS、IaaS的分层治理，各层之间通过接口互相调用，层与层之间互不侵入



容器云

分层设计

● 业务前台SaaS

- ✓ 服务于多个事业部，提供web可视化平台、api&sdk多种方式对接
- ✓ 可定制支持多种业务类型：检索业务、AI业务、大数据业务、数据库业务等等

● 服务中台PaaS

- ✓ 云管理：构建统一的云平台管理服务，管理kubernetes多集群和容器调度发布
- ✓ 云引擎：构建稳定的kubernetes+docker的高可用集群，提供高可用调度、资源隔离、弹性伸缩等特性

● 基础设施IaaS

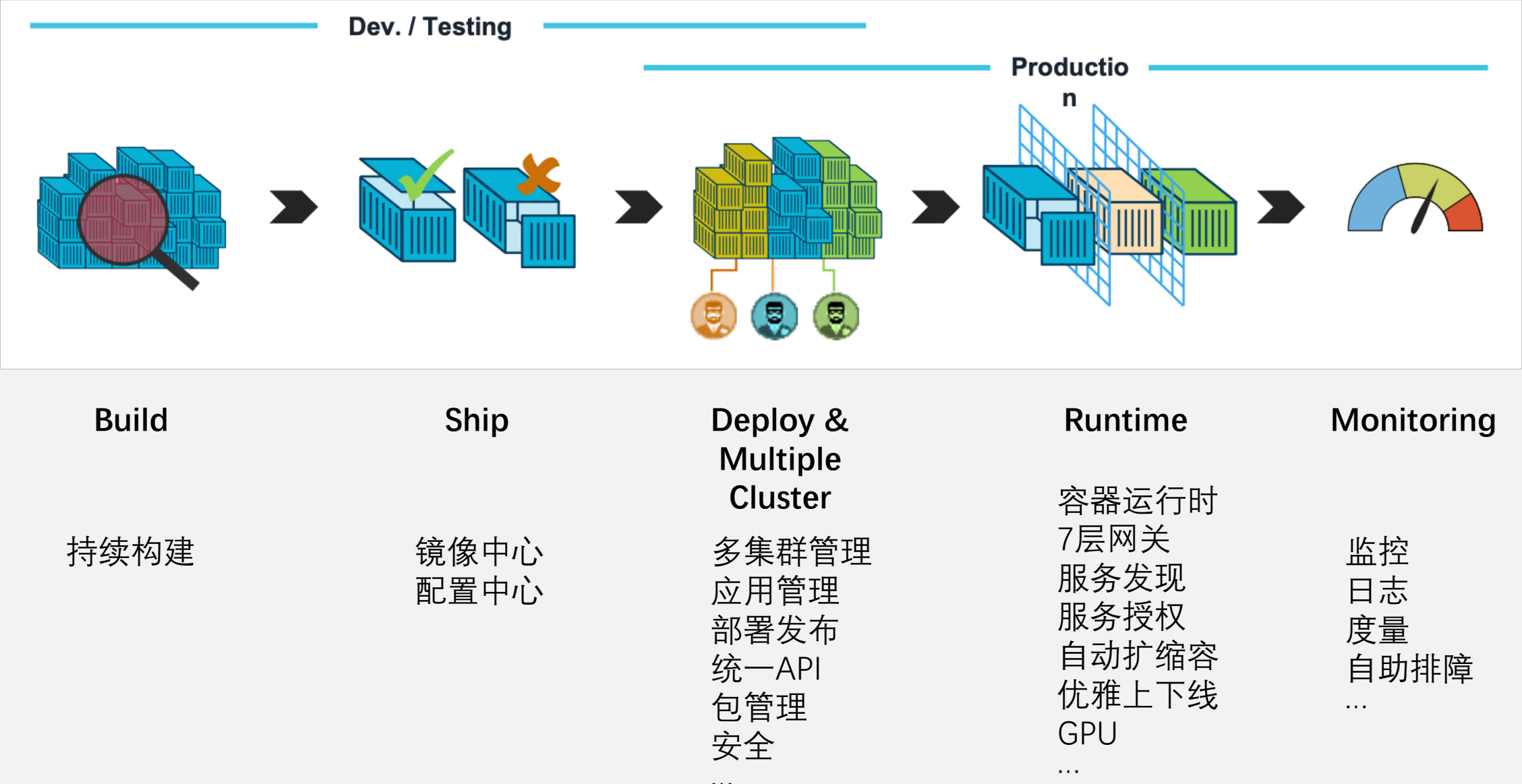
- ✓ 基础硬件资源和虚拟化资源
- ✓ 网络、分布式存储基础服务



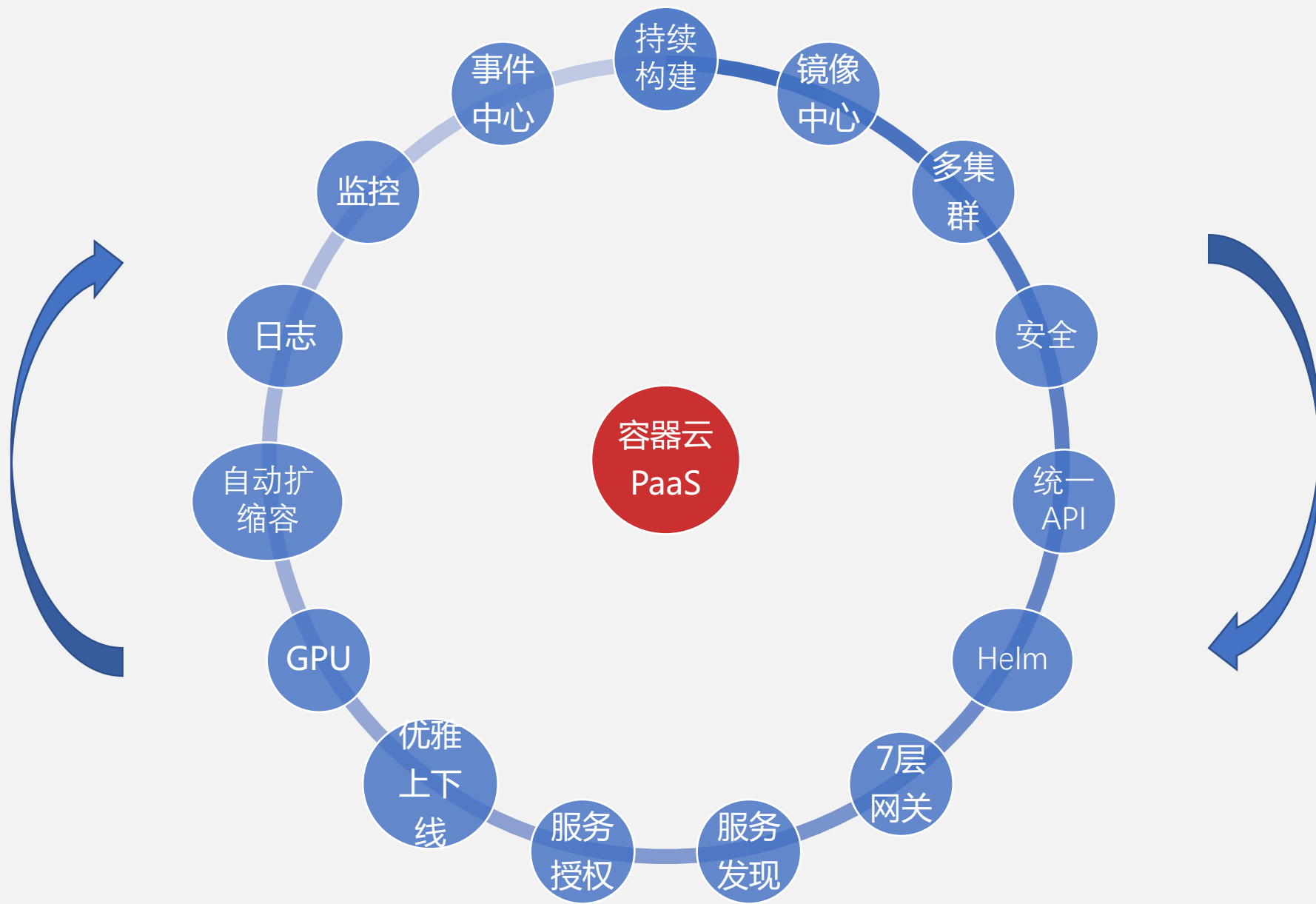
02

PaaS层设计实践

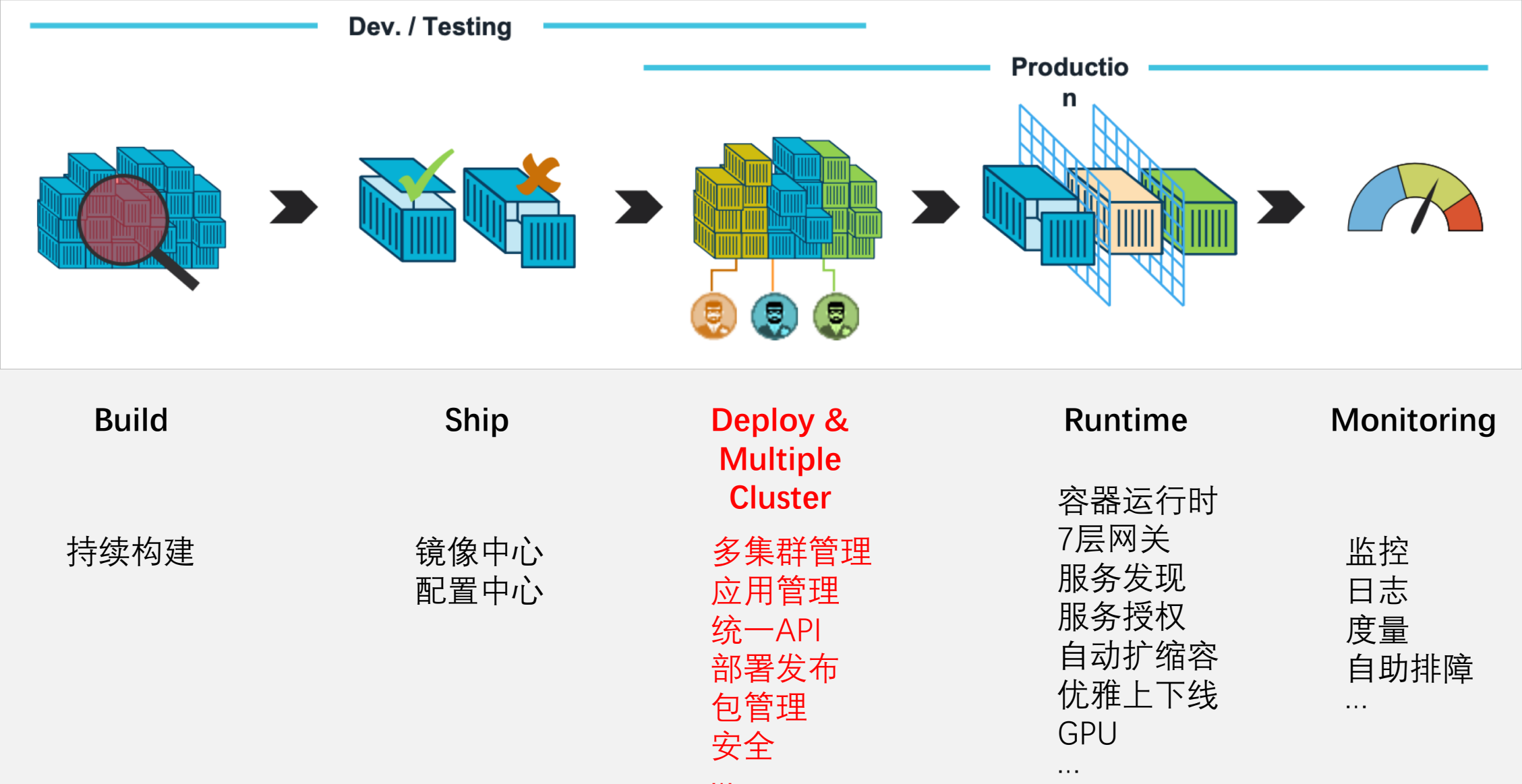
PaaS层设计实践



PaaS层设计实践

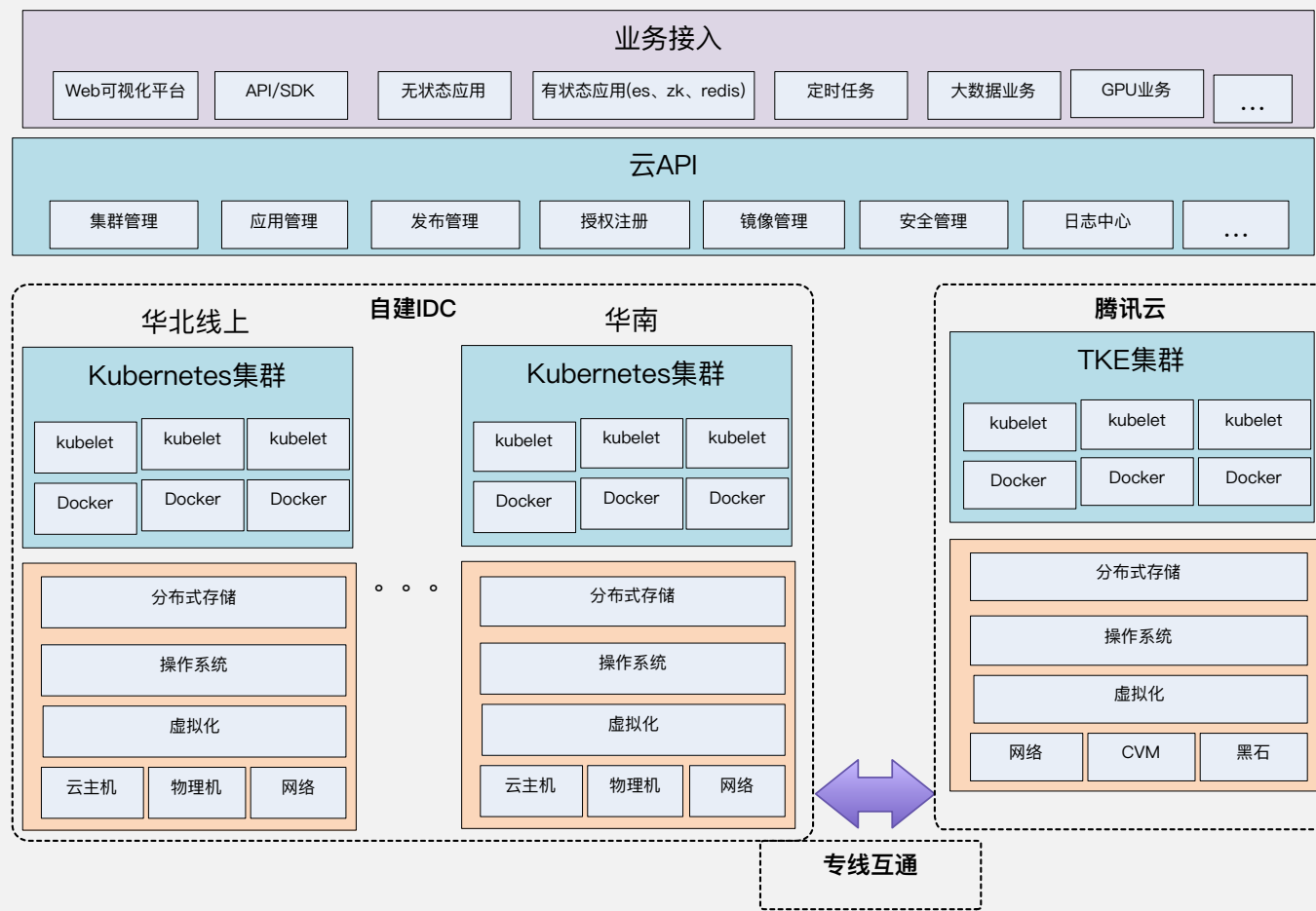


PaaS层设计实践



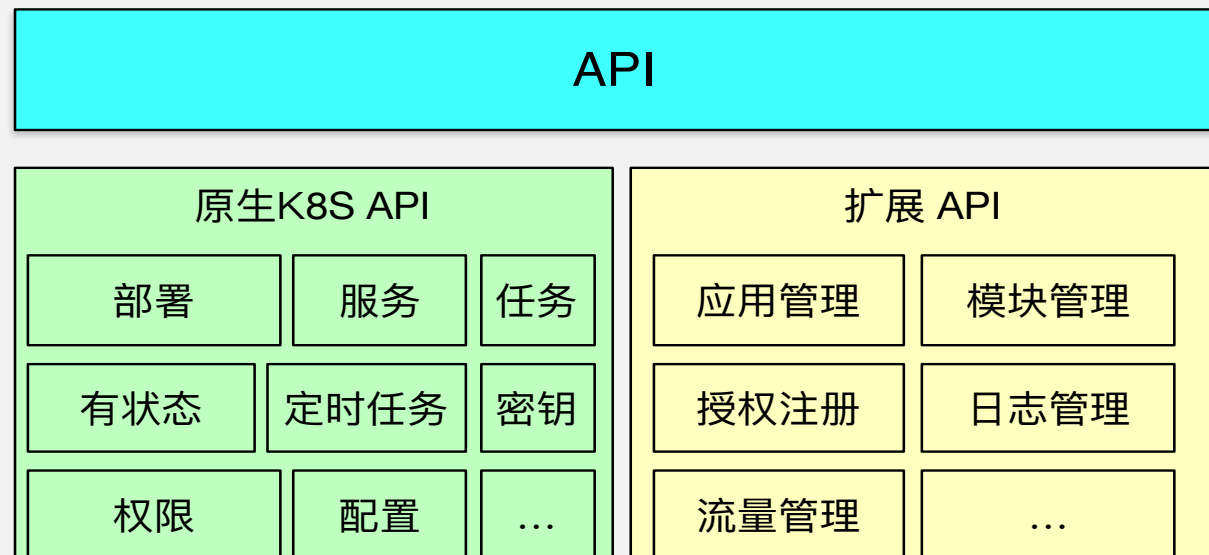
多集群

- 多集群：
 - 多环境隔离：线下实验，线上生产
 - 多地域隔离：三地域集群+腾讯TKE集群
 - 多云管理：一套roc，自由切换
- 应用
 - 应用管理：paas.sogou.com/app
 - 提供多种Workloads：无状态，有状态，凌云Job，GPU任务/虚机/服务等
- 接口统一



统一API

- 部署入口：WEB+API
- API选型：封装API OR K8S API
- K8S API的优势
 - 统一性：k8s原生API和扩展API可以保持统一
 - 成本低：开发成本远低于封装API，在部分场景下，如无需控制器功能，可以无需开发
 - 复用K8S能力：可以通过k8s CRD+Operator的能力来扩展API，可以复用RBAC来管理API权限，也可以继续使用kubectl命令行工具来进行发布



CRD

➤ API : Group + Version + Resource

- core/v1/pods(v1/pods)
- apps/v1/deployment
- batch/v1/job

Api组+版本

Api 资源

Api 元数据

Api 期望状态

Api 当前状态

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: myapp_abc
5   namespace: default
6   labels:
7     paas.sogou.com/app: myapp
8   clusterName: "myapp"
9 spec:
10  hostNetwork: true
11  dnsPolicy: ClusterFirstWithHostNet
12  #dnsPolicy: Default
13  containers:
14  - name: myapp-container
15    image: qa.registry.biz.sogou.biztech/biz_centos7.2:latest
16    command: ["/bin/sh"]
17    args: ["-c", "while true; do echo hello; sleep 10;done"]
18    #securityContext:
19    # privileged: true
```

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginxtest_abc
5   namespace: thanos
6   clusterName: "mytest"
7   labels:
8     pass.sogou.com/app: nginxtestapp
9 spec:
10  replicas: 1
11  strategy:
12    rollingUpdate:
13      maxSurge: 1
14      maxUnavailable: 1
15  type: RollingUpdate
16  revisionHistoryLimit: 25
17  selector:
18    matchLabels:
19      app: nginxtest
```

CRD

➤ CustomResourceDefinition(CRD) :
apiextensions.k8s.io/customresourcedefinitions/v1beta1

CRD名称 : <plural>.<group>

CRD组 : <group>

CRD资源名

CRD命名空间范围

CRD版本

```
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
metadata:
  name: nginxvhosts.paas.sogou.com
spec:
  group: paas.sogou.com
  names:
    kind: NginxVhost
    listKind: NginxVhostList
    plural: nginxvhosts
    singular: nginxvhost
  scope: Namespaced
  version: v1
  validation:
```

CRD

创建CR：CRD创建成功后，可以创建自定义资源(CustomResource，CR)

➤ CR Struct：

- ✓ TypeMeta(固定)
- ✓ ObjectMeta(固定)
- ✓ Spec(自定义)
- ✓ Status(自定义)

➤ RestfulAPI：

apis/paas.sogou.com/v1/NginxVhosts

Operator：《深入理解K8S Controller》

```
apiVersion: paas.sogou.com/v1
kind: NginxVhost
metadata:
  creationTimestamp: "2019-10-11T09:45:54Z"
  finalizers:
    - paas.sogou.com/thanos
  generation: 43758
  labels:
    paas.sogou.com/app: bizcloud-api
    paas.sogou.com/refer: bizcloud
  name: bizcloud-api-vhost-pre
  namespace: thanos
  resourceVersion: "92939011"
  selfLink: /apis/paas.sogou.com/v1/namespaces/thanos/nginxvhosts/bizcloud-api-vhost-pre
  uid: eb0fca4e-ec0b-11e9-8c61-246e9637ae5c
spec:
  domain: pre.api.thanos.sogou
status:
  message: ""
  status: Created
  updateTime: "2019-12-05T21:19:01+08:00"
```

TypeMeta

ObjectMeta

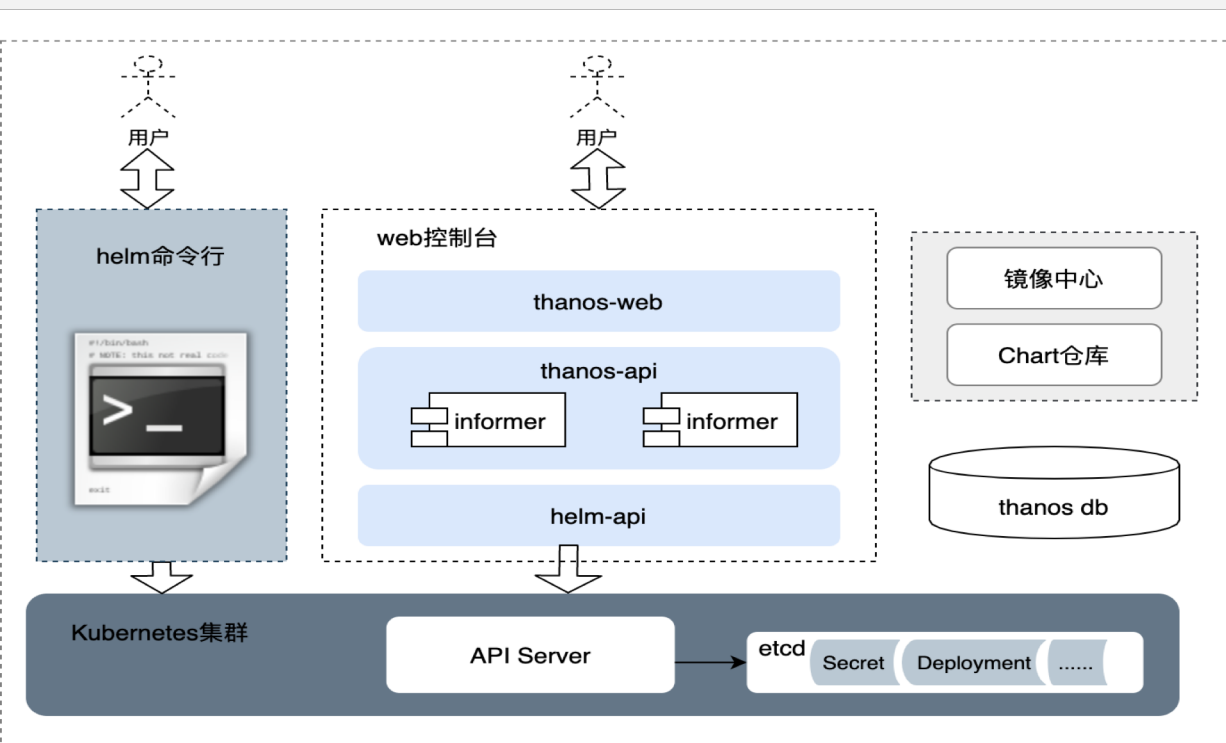
Spec

Status

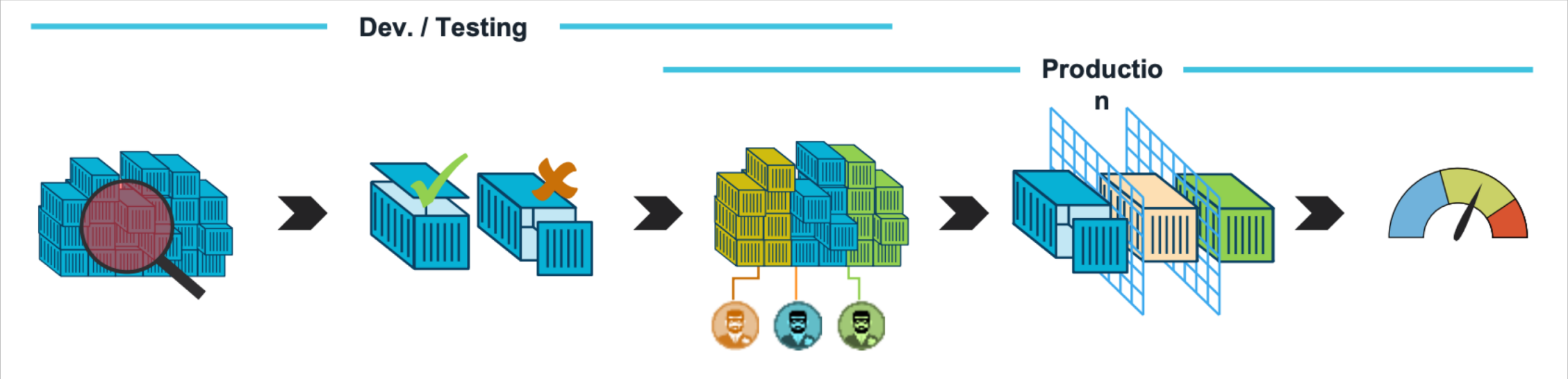
Helm发布

- 容器云也集成了HELM包管理工具：

- ✓ 提供命令行和Web方式：根据Helm v3源码开发出web服务，提供标准的restful api接口调用
- ✓ 提供多套公有helm模板，支持自定义模板



PaaS层设计实践



Build

持续构建

Ship

镜像中心
配置中心

Deploy &
Multiple
Cluster

多集群管理
应用管理
统一API
部署发布
包管理
安全

Runtime

容器运行时
7层网关
服务发现
服务授权
扩缩容
优雅上下线
GPU

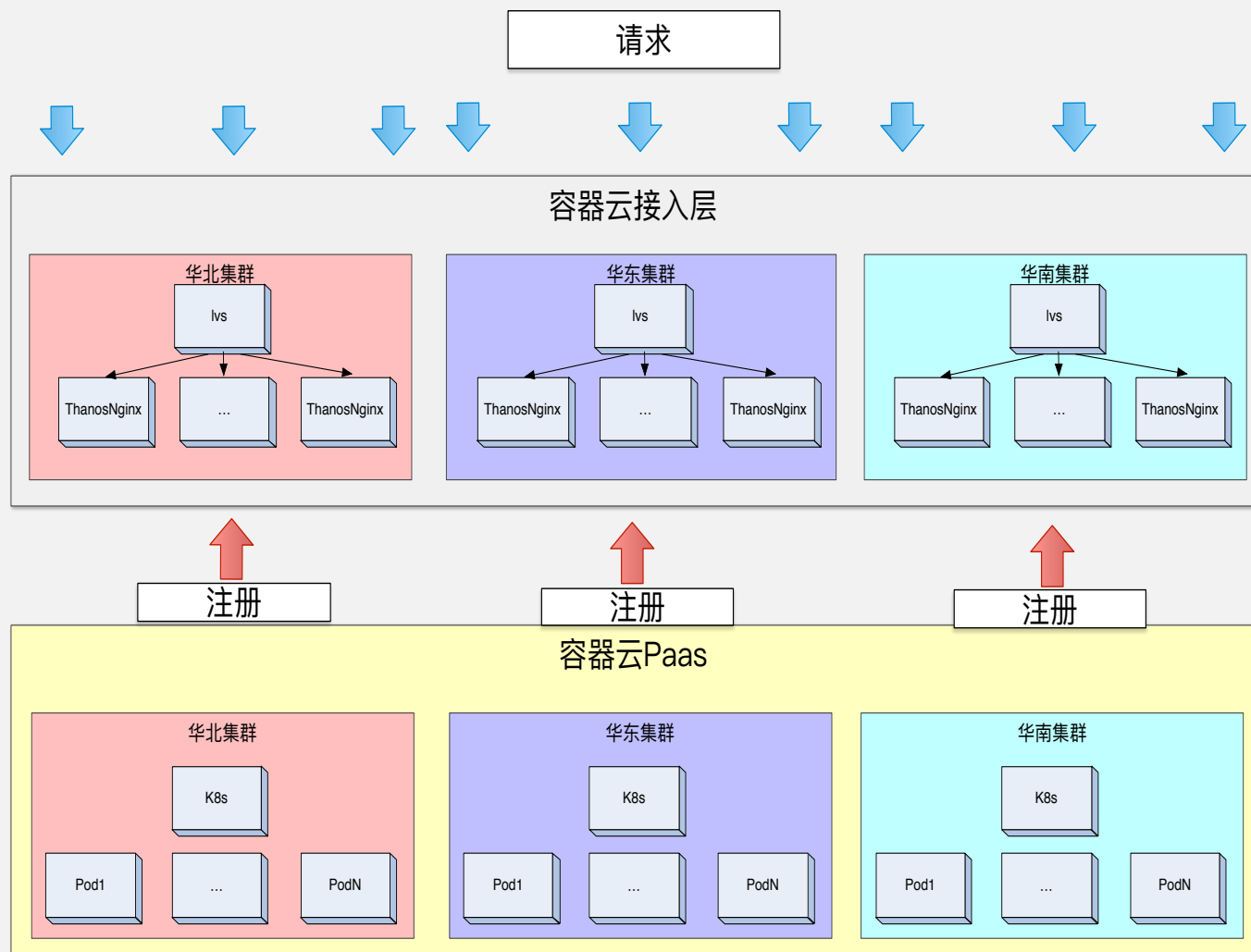
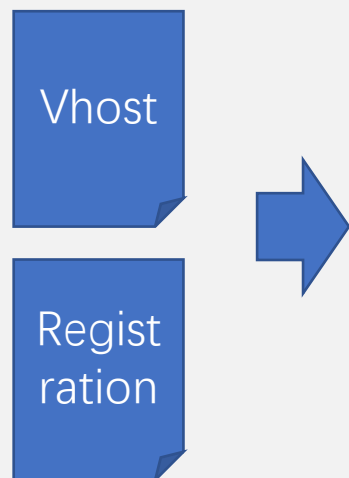
Monitoring

监控
日志
度量
自助排障
...

7层网关

- 目标：

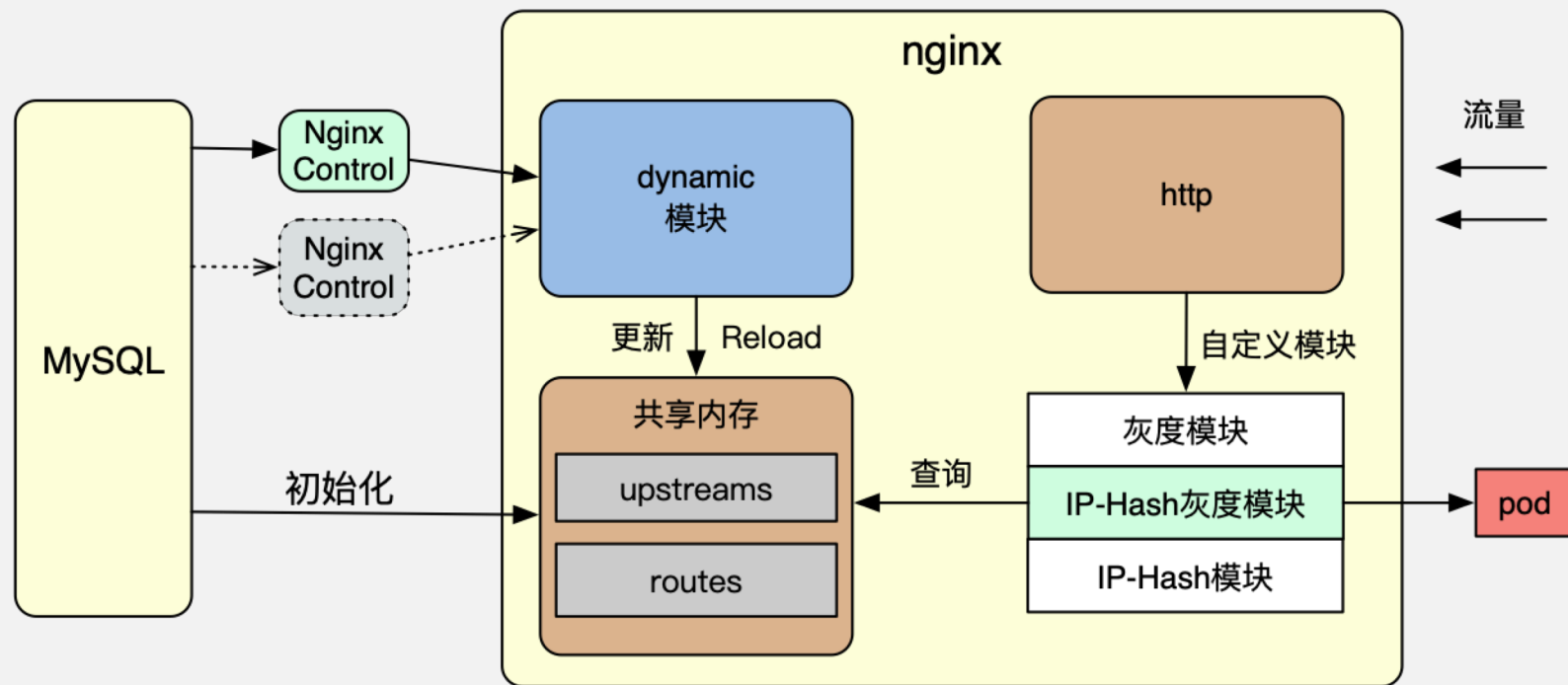
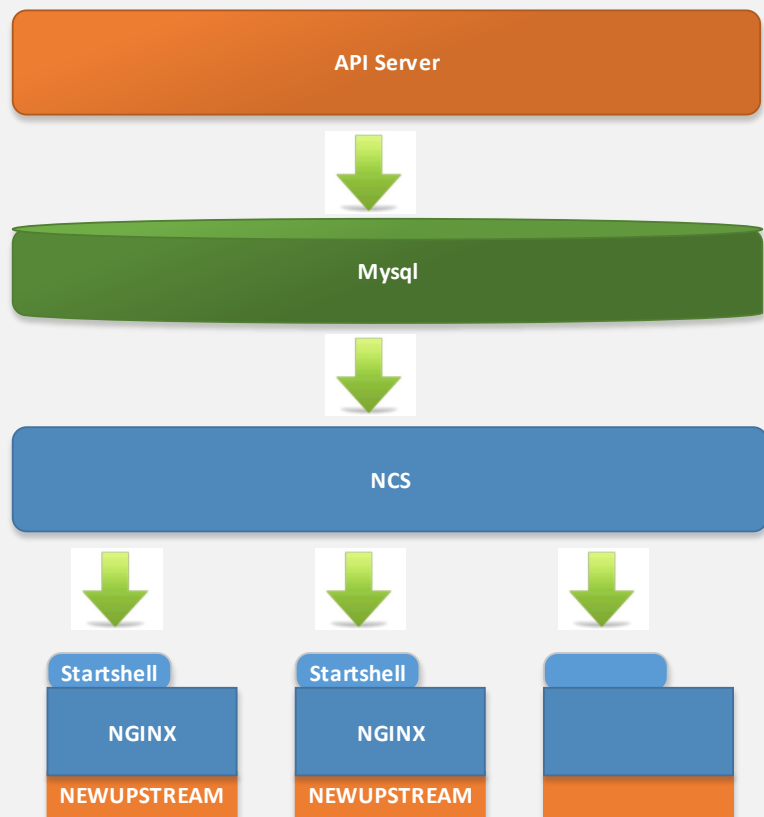
- ✓ 实现的统一的7层流量接入，提供反向代理能力
- ✓ 服务配置秒级实时动态热更新：容器IP易变性
- ✓ 提供流量管理能力：蓝绿发布、灰度发布
- ✓ 稳定性保障



7层网关

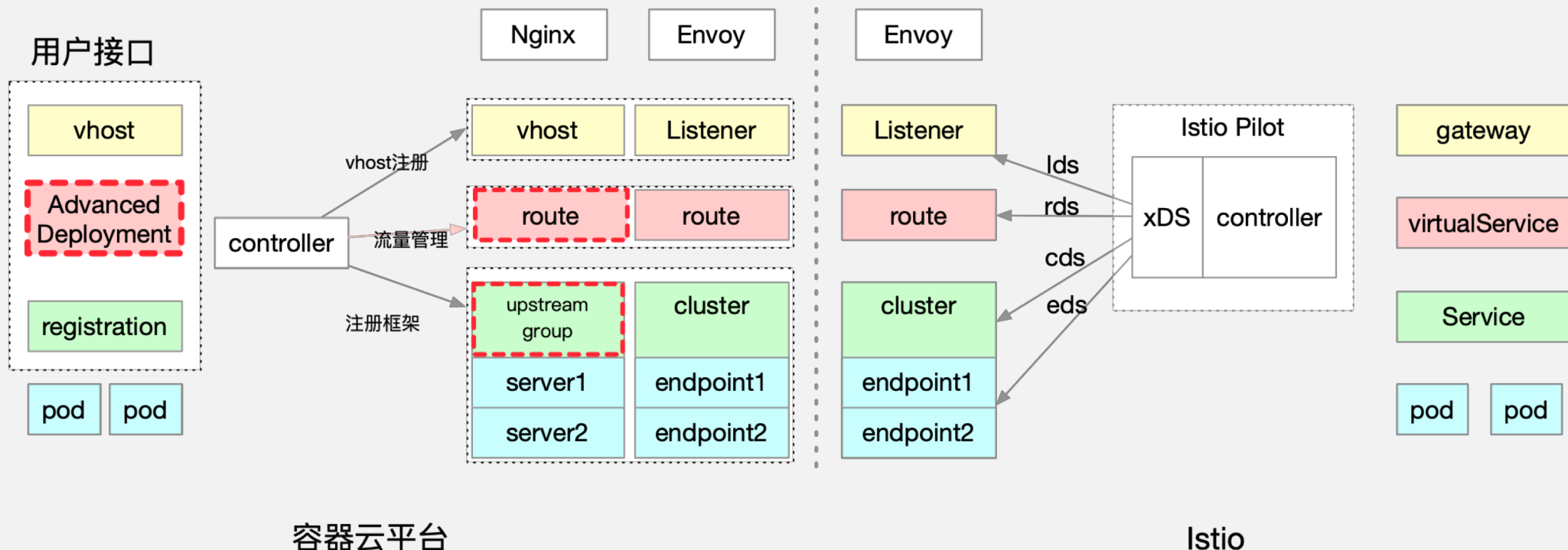
- 配置秒级实时动态更新：

- ✓ 实时同步：K8S Watch(注册框架) + Mysql binlog的同步链路，实现秒级更新
- ✓ 热更新：Nginx Dynamic Upstream，降低90%的reload



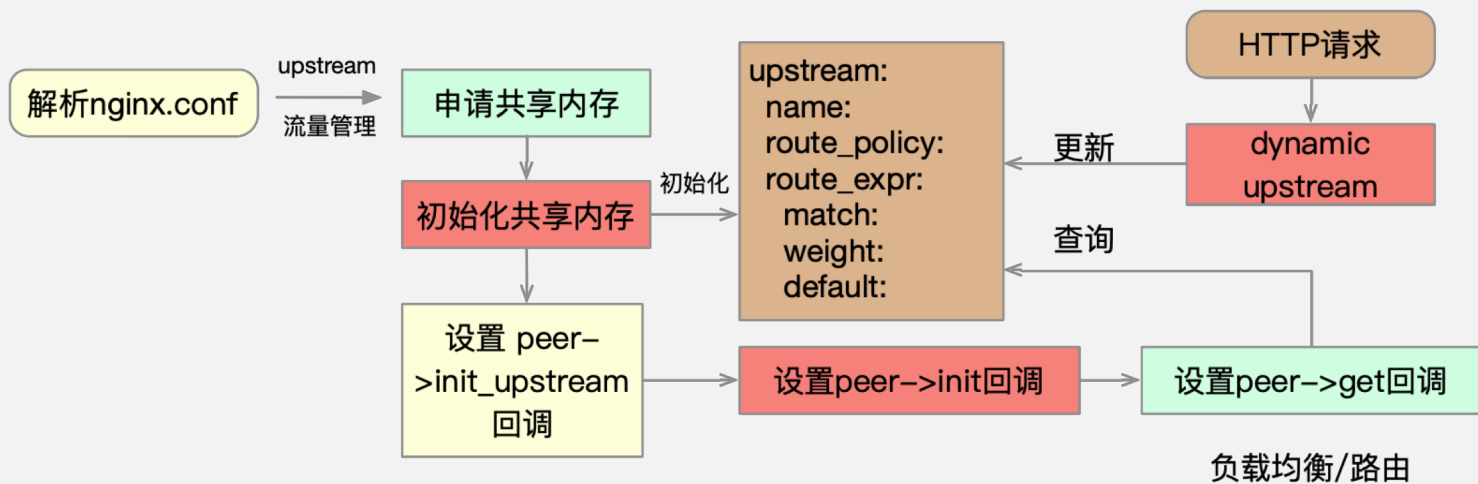
7层网关

- 流量管理：
 - ✓ 提供蓝绿、灰度发布，以及灵活的流量控制策略
 - ✓ 兼容Nginx和Envoy代理，支持无缝切换
 - ✓ 提供4层、GRPC代理
 - ✓ 提供SRPC代理



7层网关

Nginx路由策略维护



• 流量管理：

✓ 自定义路由：

✓ Match：根据请求内容匹配

✓ Weight：根据流量比例进行分组

✓ 实现方式：

✓ Nginx扩展Upstream能力：增加路由策略、分组信息

✓ Envoy通过RDS动态调整Route策略

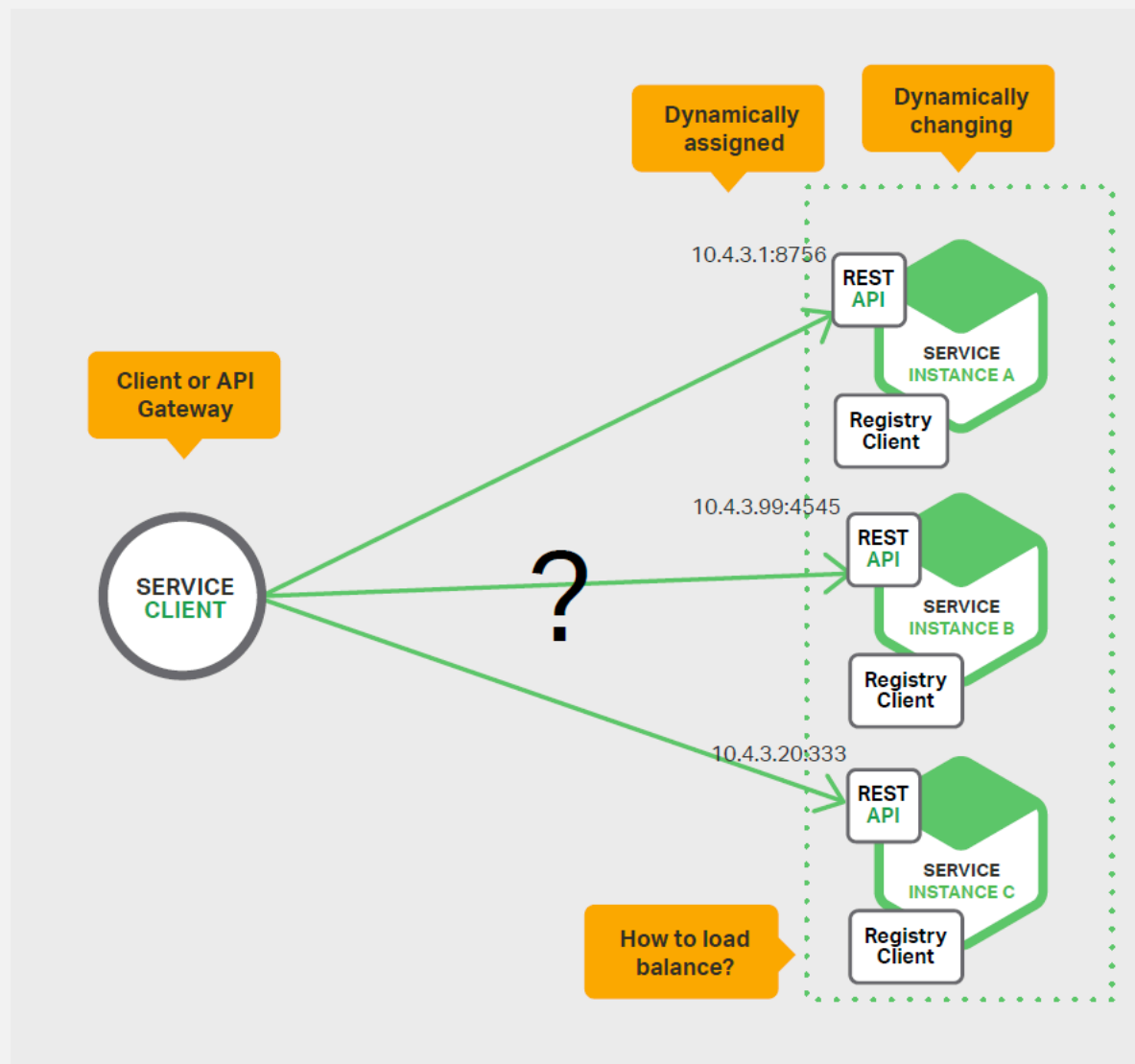
• 稳定性保障：

✓ 多方位监控体系：基于智子监控+内存状态比对等多个监控手段，保证接入层的稳定运行

✓ 度量体系：建设了围绕接入层多个指标的度量，方便数据跟踪，问题定位

服务发现

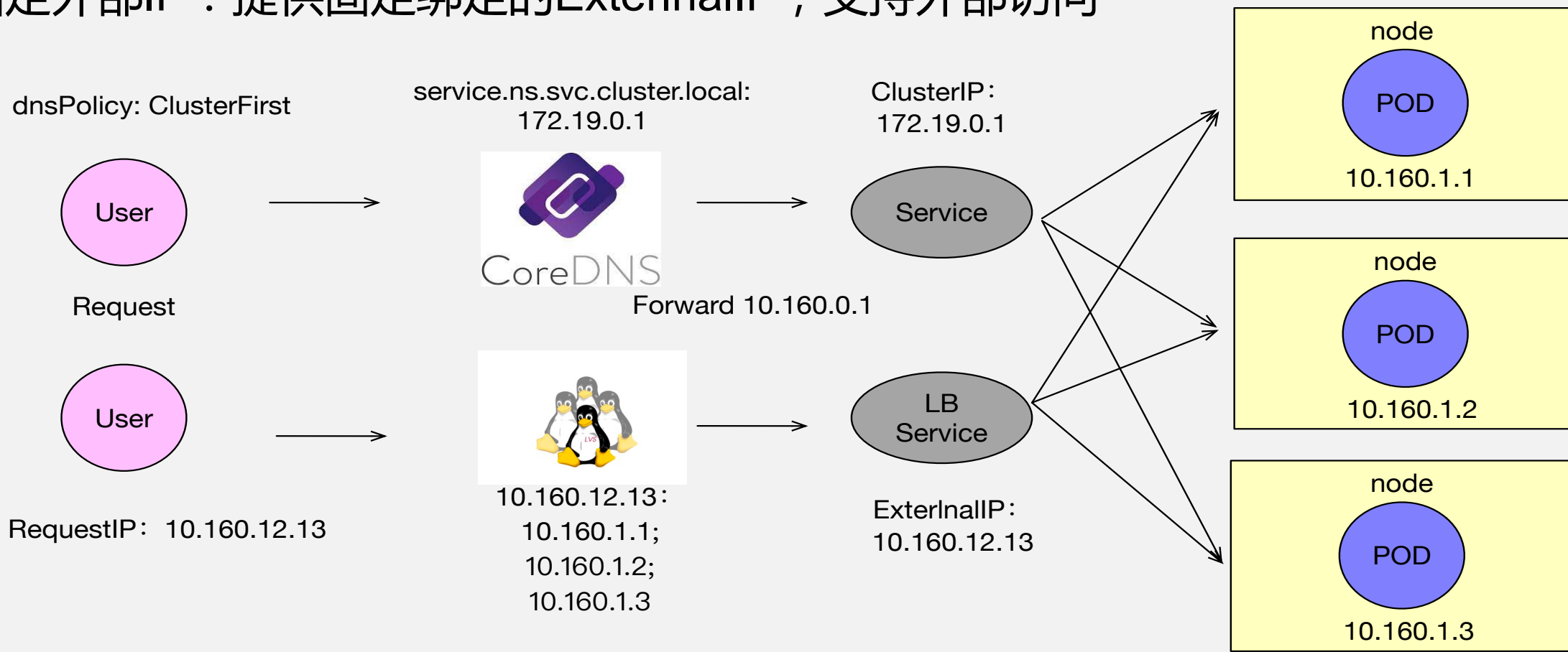
- 4层的服务发现能力：
 - ✓ DNS
 - ✓ 统一注册框架



服务发现

➤ DNS :

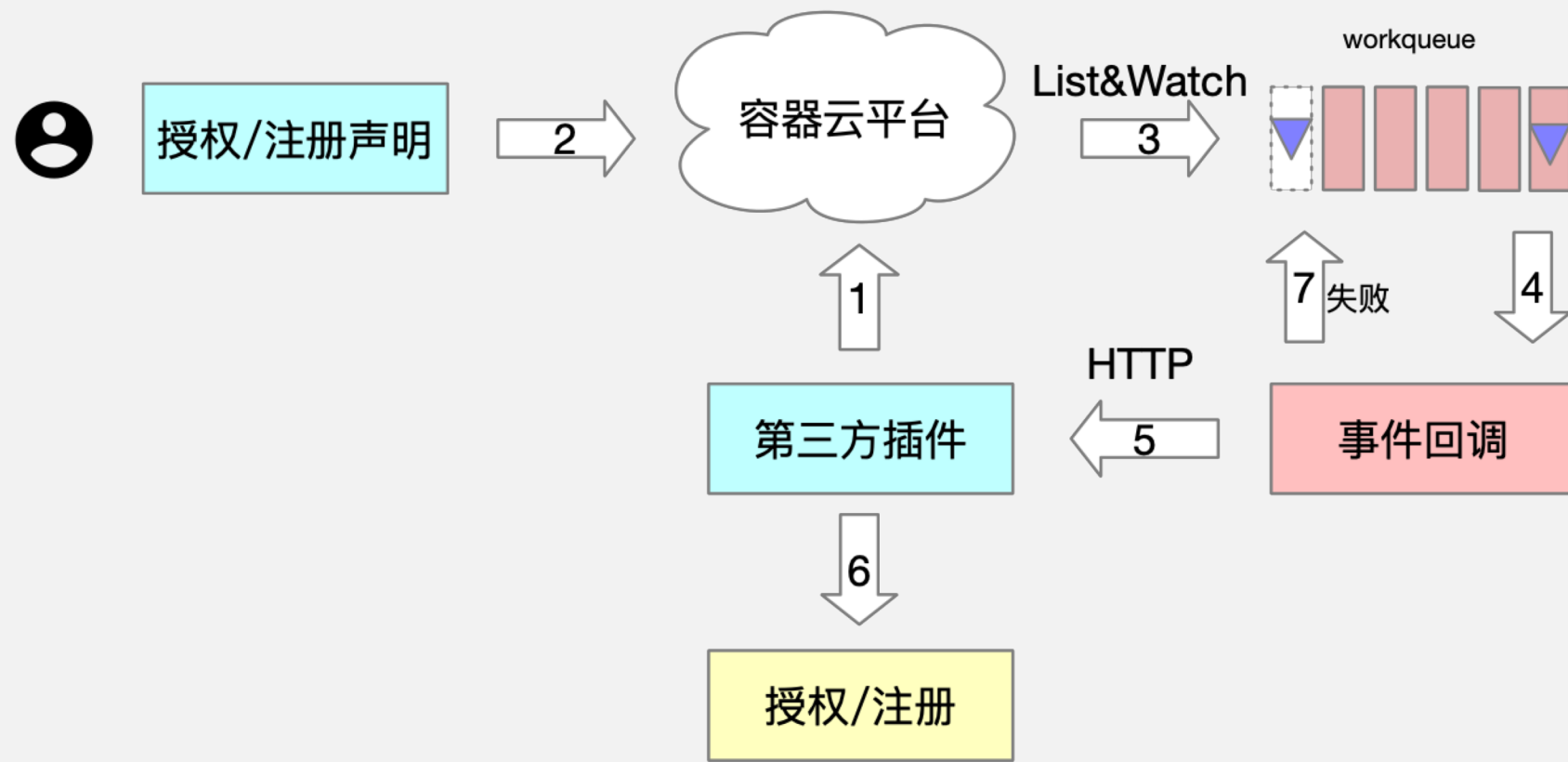
- ✓ 内部域名自动生成：通过Service自动生成集群内域名
- ✓ 外部域名支持：Forward公司域名，支持.sogou等外部域名解析
- ✓ 固定外部IP：提供固定绑定的ExternalIP，支持外部访问



服务发现

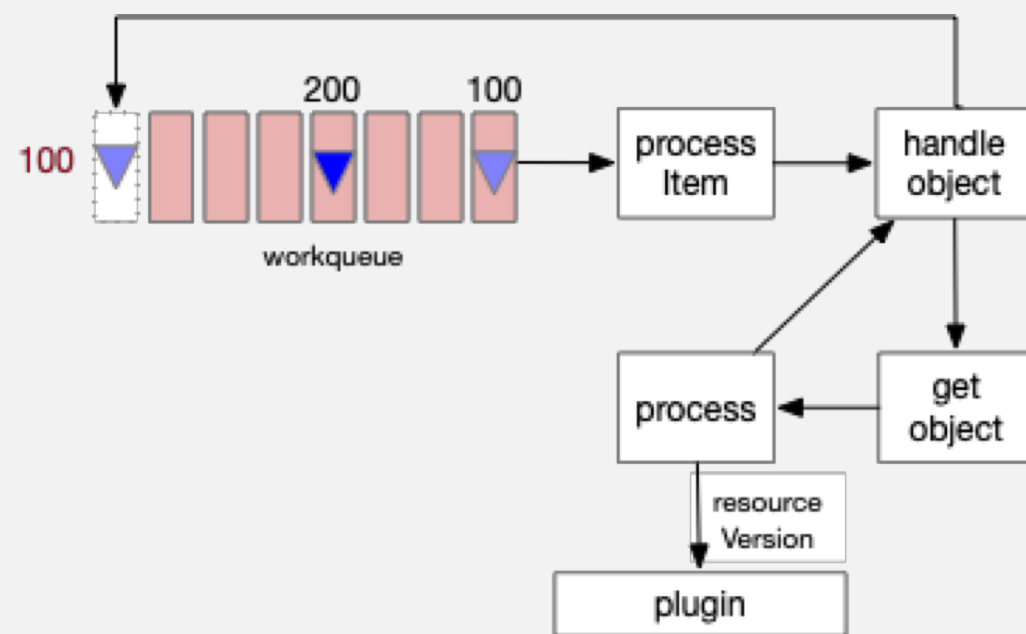
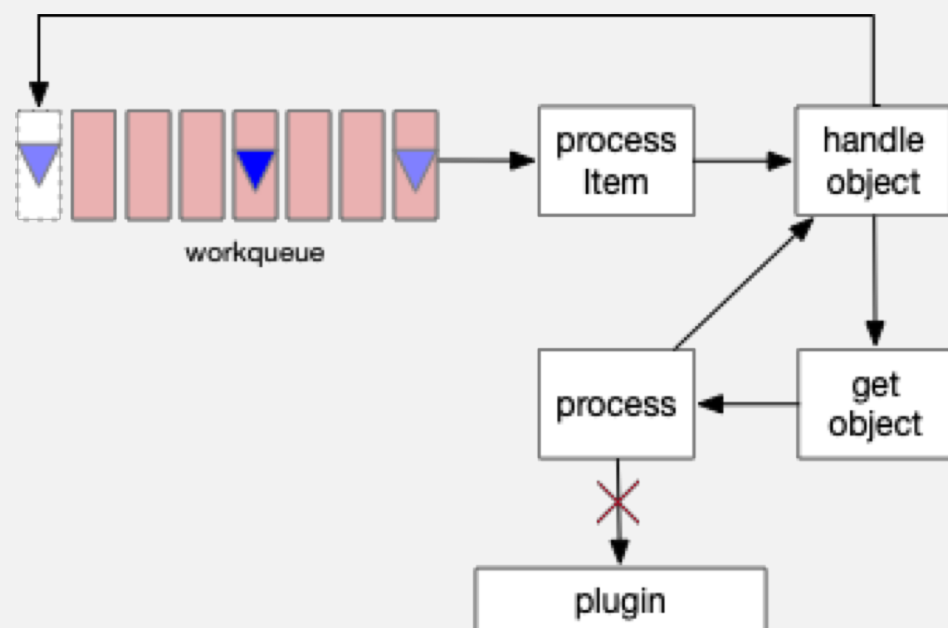
➤ 统一注册框架：

- ✓ 插件化：下游注册接口可以插拔式接入框架，无需感知k8s细节
- ✓ 实时保序推送：实时监听容器状态变更，框架保证pod级保序推送
- ✓ 多状态感知：如PodReady感知，除了PodIP变更感知，还可以感知pod ready状态



服务发现

➤ 实践遇到的挑战：



服务发现

➤ 实践遇到的挑战：

spec:

actions:

- server: nginx
operation: apply
attributes:
- server: soa
operation: apply
attributes:
- server: biztree
operation: revoke
attributes:
- server: nginx
operation: apply
attributes:

status:

- status: applyFailed
- status: applySuccess
- status: revokeSuccess
- status: applySuccess

阻塞&退出：影响其他插件

忽略：乱序风险



nginx



soa



biztree



nginx

有序
调用

写回状态

插件级并行

插件内串行，阻塞&退出

nginxList



nginx

soaList



soa

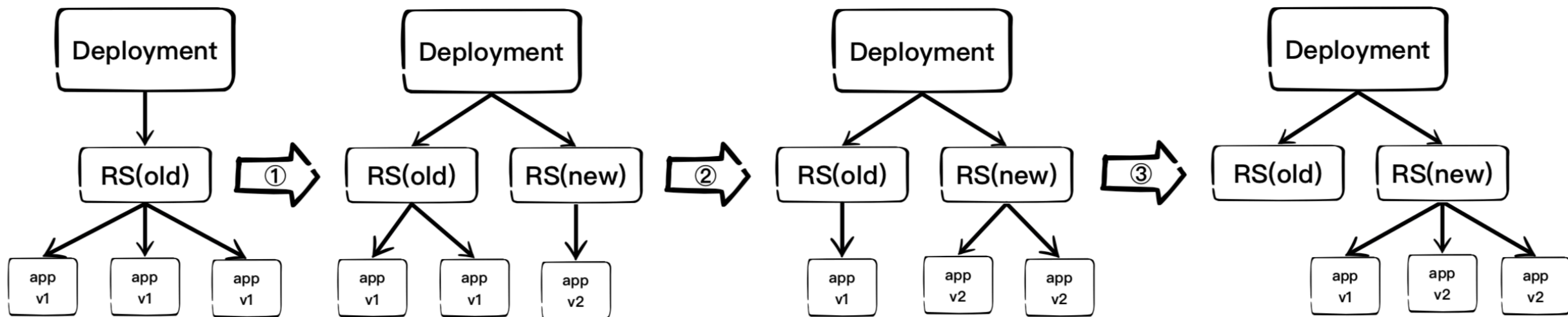
biztreeList



biztree

优雅上下线

➤滚动升级



➤滚动升级的业务高可用

- 新POD上线，IP已经注册完成，但节点还未Ready，导致访问失败
- 旧POD下线，容器已经完成回收，但节点还并未注册下线，导致访问失败

优雅上下线

➤ Container Lifecycle

- 上线过程：如何保证先容器Ready再注册
- 下线过程：如何保证先反注册再下容器

➤ 常规方案：

- 在ReadinessProbe增加注册脚本
- 在Pre stop hook增加等待脚本

➤ 优缺点

- 优点：直观易懂
- 缺点：对用户侵入大，不优雅；不能精准控制



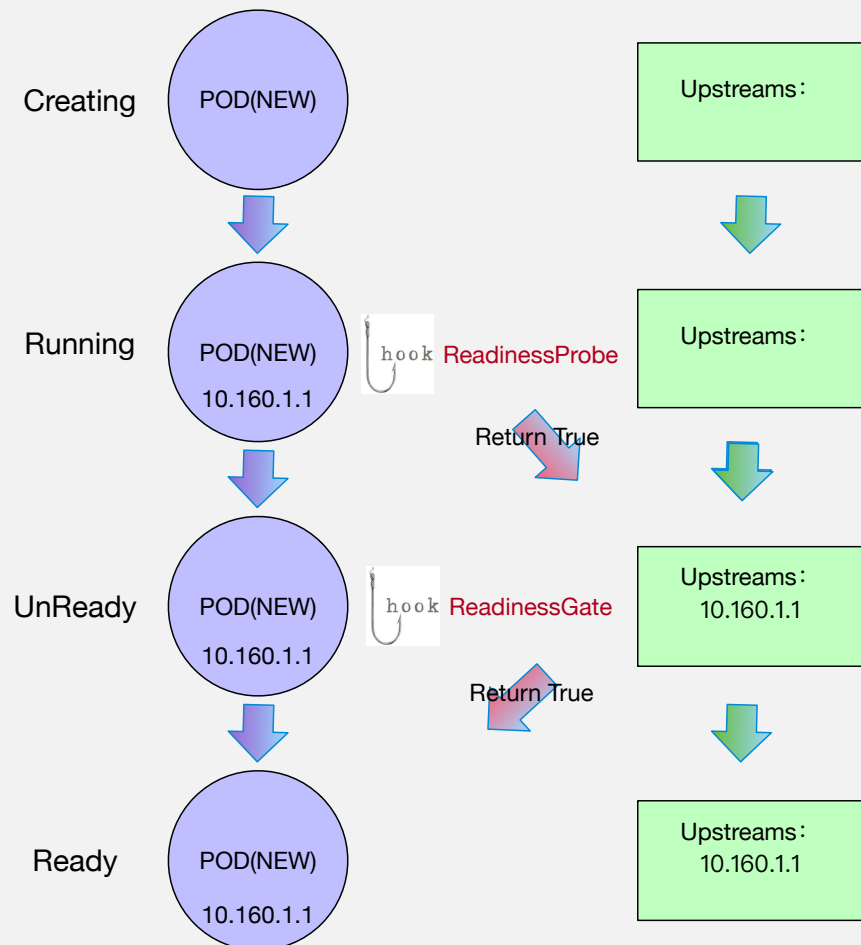
```
lifecycle:
preStop:
  exec:
    command:
      - sleep
      - "60"
```

优雅上下线

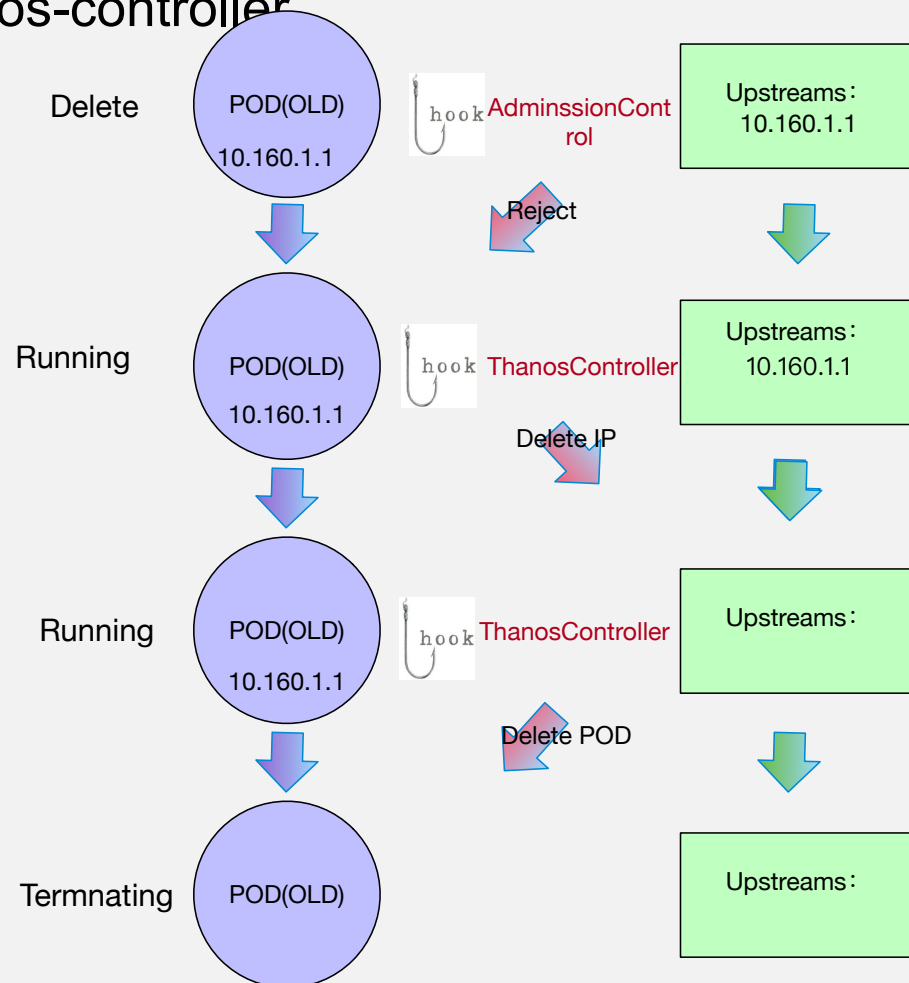
➤ 容器云平台的解决方案：优雅上下线(用户基本无感知)

- 优雅上线：引入超级探针，使用readinessProbe + readinessGate
- 优雅下线：引入准入控制，使用admissionControl + thanos-controller

优雅
上线

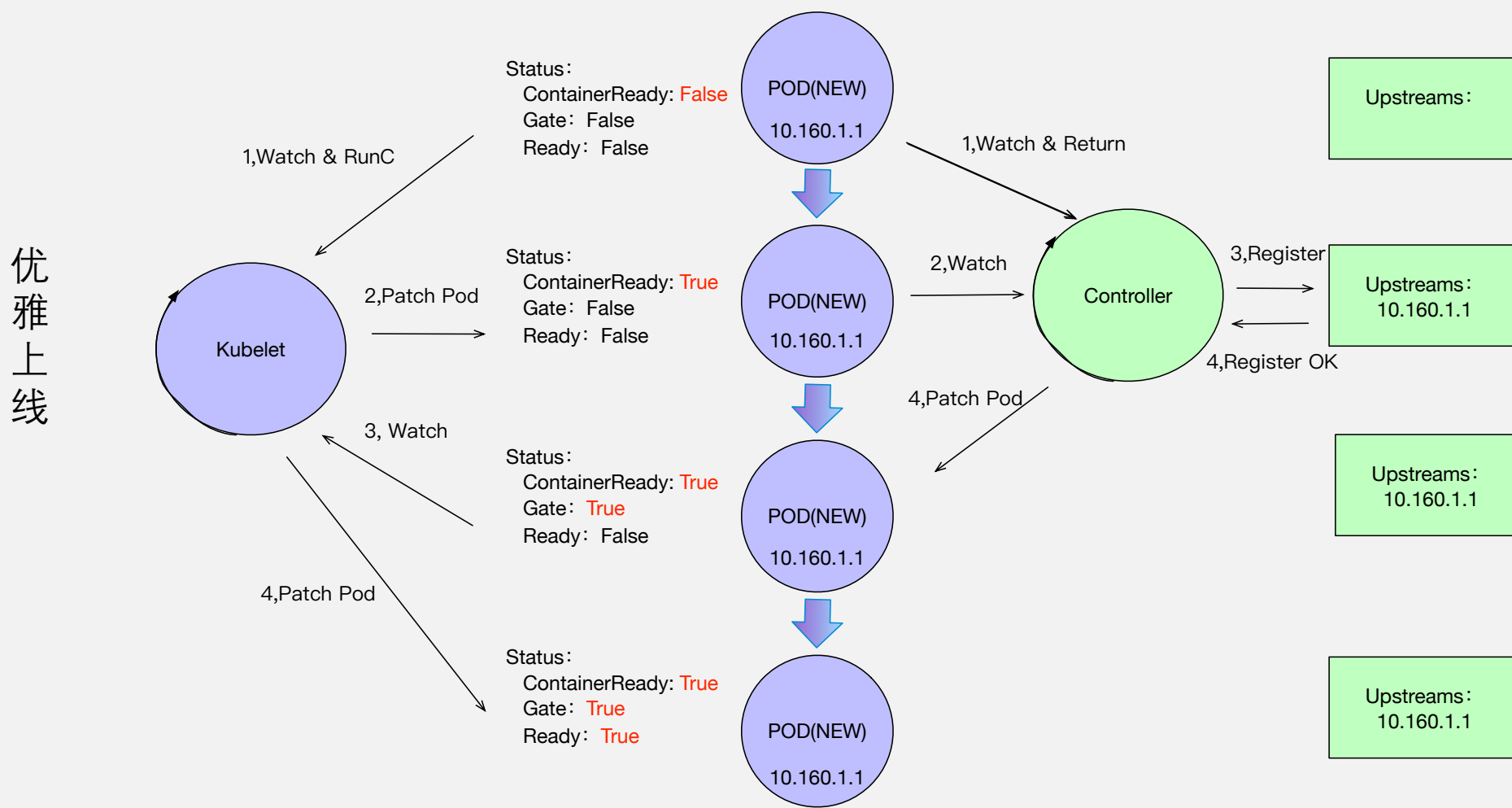


优雅
下线

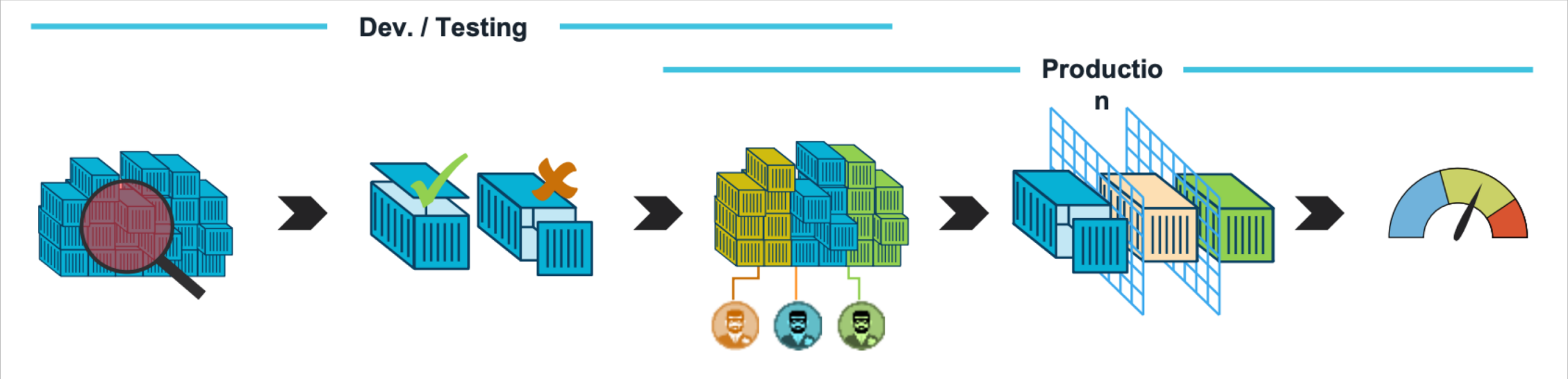


优雅上下线

➤优雅上线的详细流程：



PaaS层设计实践



Build

持续构建

Ship

镜像中心
配置中心

Deploy &
Multiple
Cluster

多集群管理
应用管理
统一API
部署发布
包管理
安全

Runtime

容器运行时
7层网关
服务发现
服务授权
自动扩缩容
优雅下线
GPU
...

Monitoring

监控
日志
度量
自助排障
...

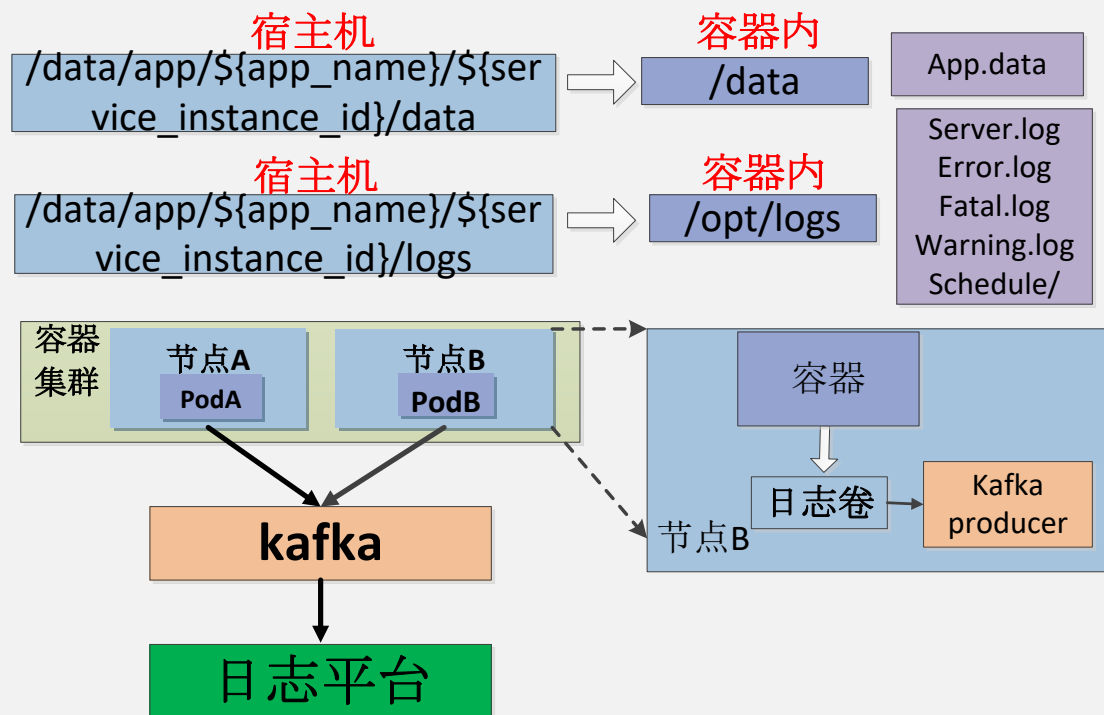
日志中心

➤设计目标：

- ✓实现统一的，标准化的容器日志中心
- ✓提供更多挖掘日志价值能力的平台和工具
- ✓解决容器漂移等复杂场景情况下日志采集问题

➤常见日志场景：

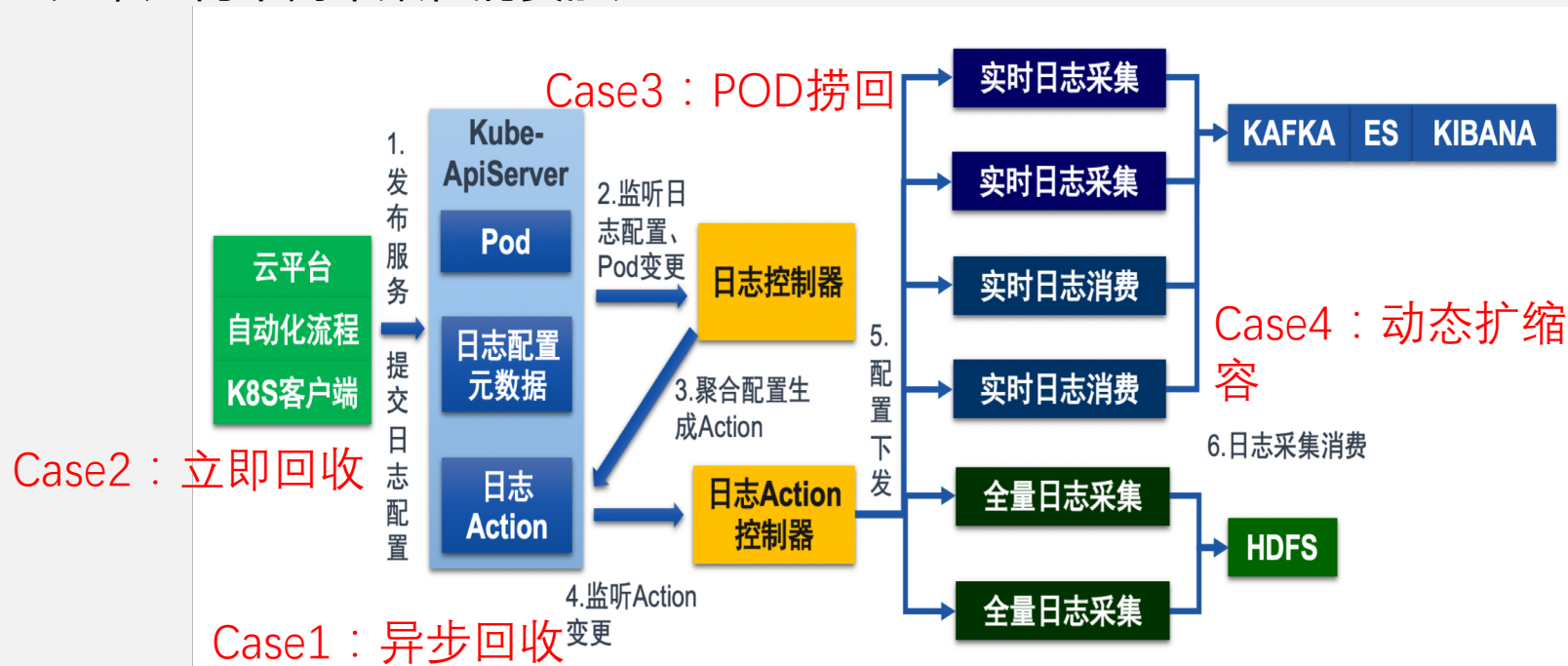
- ✓日志模式：实时日志+全量日志备份
- ✓日志格式：文件+标准输出
- ✓日志挂载：HostPath，EmptyDIR，PVC



日志中心

➤ 日志中心架构

- 平台构建：控制端+采集端(全量+实时)+Kafka+ELK
 - 采集端：daemonset+sidecar
- 复杂场景：
 - ✓ Case1：线上应用的POD遇到异常终结漂移时，是否能立即回收日志采集配置
 - ✓ Case2：当用户撤销日志采集时，是否能立即回收日志采集配置
 - ✓ Case3：POD运行回收和日志配置下发是并行的过程，在POD快速回收的场景下如何获取POD的日志挂载数据。
 - ✓ Case4：不同应用的日志量差别巨大，如何平衡采集和消费能力



自助排障

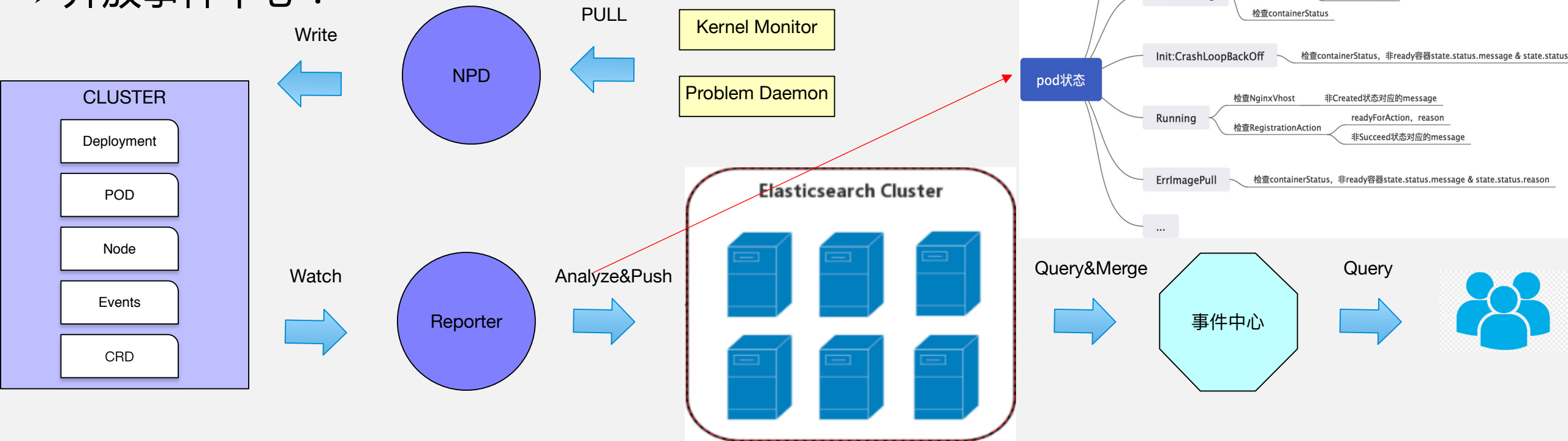
➤自助排障：随着应用接入规模的逐渐增大，加上容器环境保留困难的特点，排障的成本也越来越高

- 我的容器为什么发生重启了，什么时候，什么原因重启的？
- 我的容器为什么一直起不来，为什么一直在init状态，为什么一直是UnReady？



➤WebShell

➤开放事件中心：



事件中心

➤Step1：查询历史Pod的历史状态分析

Pod历史

帮助文档

命名空间:

thanos

应用:

test-oom

过去30分钟

发生时间	PodName	Pod状态	PodIP	宿主IP	所属应用	原因	描述	操作
2020-12-24 10:18:24	test-oom-7bf957f48d-7qtdp	Terminating	10.143.40.101	10.143.40.101	test-oom			关联事件 日志
发生时间	PodName	Pod状态	PodIP	宿主IP	所属应用	原因	描述	
2020-12-24 10:18:24	test-oom-7bf957f48d-7qtdp	Terminating	10.143.40.101	10.143.40.101	test-oom			
2020-12-24 10:18:15	test-oom-7bf957f48d-7qtdp	Error	10.143.40.101	10.143.40.101	test-oom			
2020-12-24 10:18:13	test-oom-7bf957f48d-7qtdp	Running	10.143.40.101	10.143.40.101	test-oom			
2020-12-24 10:18:12	test-oom-7bf957f48d-7qtdp	CrashLoopBackOff	10.143.40.101	10.143.40.101	test-oom			
2020-12-24 10:18:00	test-oom-7bf957f48d-7qtdp	Error	10.143.40.101	10.143.40.101	test-oom			
2020-12-24 10:17:59	test-oom-7bf957f48d-7qtdp	Running	10.143.40.101	10.143.40.101	test-oom			

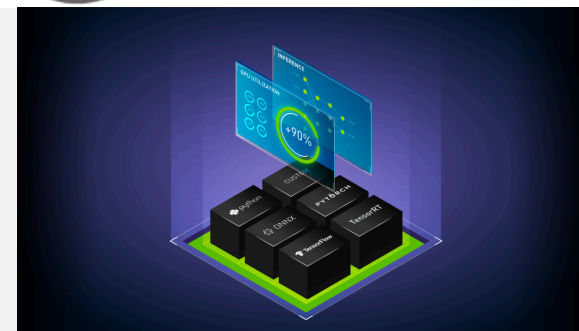
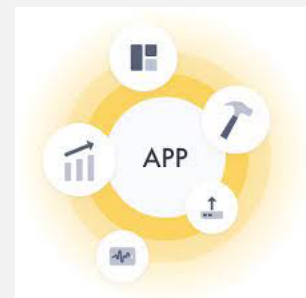
➤Step2：关联事件查询进行根因分析

关联事件

发生时间	事件等级	对象类型	对象名称	原因	描述
2020-12-24 10:18:14	Warning	Pod	test-oom-7bf957f48d-7qtdp	BackOff	Back-off restarting failed container
2020-12-24 10:18:13	Warning	Node	10.143.40.101	OOMKilling	Memory cgroup out of memory: Kill process 3933802 (k8s-monitor) score 138 or sacrifice child Killed process 3933802 (k8s-monitor) total-vm:872712kB, anon-rss:86552kB, file-rss:19760kB, shmem-rss:0kB
2020-12-24 10:18:11	Normal	Pod	test-oom-7bf957f48d-7qtdp	Started	Started container cache-searchhub
2020-12-24 10:18:11	Normal	Pod	test-oom-7bf957f48d-7qtdp	Created	Created container cache-searchhub
2020-12-24 10:18:11	Normal	Pod	test-oom-7bf957f48d-7qtdp	Pulled	Container image "docker-reg.sogou-inc.com/thanos-controller/k8s-monitor:0.0.1-0" already present on machine

其他实践

- 持续构建
- 镜像中心
- 应用管理
- 安全、权限管理
- GPU平台
- 自动、定时扩缩容
- 监控中心
- 度量
- 服务网格
- ...



Grafana



Prometheus

03

总结

总结

回顾：

- ✓ 容器云的背景和构建原则
- ✓ PaaS层部分功能的设计与实践
 - ✓ BUILD
 - ✓ SHIP
 - ✓ DEPLOY
 - ✓ RUNTIME
 - ✓ MONITOR

总结

- 机遇：
 - ✓ 全面拥抱云原生
 - ✓ 创造价值
 - ✓ 合作共赢
- 挑战：
 - ✓ 大规模实践
 - ✓ 混布调度
 - ✓ 推广接入



总结

- 展望：

- ✓ 多云管理：实现多集群统一部署和调度
- ✓ 统一的服务发现(腾讯北极星)
- ✓ 7层网关：
 - 持续的稳定性、性能保障(150W+QPS)
 - 同步链路实时性保证
 - 丰富负载均衡策略
 - 规避极端场景下的流量损失
- ✓ 日志中心：提升采集、消费的性能；丰富日志聚合、分析的能力
- ✓ 根因分析：事件中心提供根因分析能力，直接给出原因，缩短排障路径
- ✓ 可度量：丰富数据看板，提供更精细化指标
- ✓ 更多工作负载：接入更多不同类型workload
- ✓ ...



总结

公司容器云平台(灭霸)：

线上地址：<http://roc.sogou/new/app/operation/thanos>

用户手册：<http://thanos-doc.thanos.sogou/docs/>





谢谢